QATIPv3 – Azure Lab5b
Create an Application Gateway on Azure using Terraform

Contents

Lab Objectives

In this lab, you will:

1. Review resources provisioned using modularized code. These include a resource group, network stack and virtual machines.
2. Deploy additional resources by writing code that references outputs from the network module. An application gateway will be used to distribute traffic submitted to a public IP address across backend virtual machines. A nic association will be used to associate the virtual machine nics with application gateway.

Teaching Points

This lab aims to demonstrate how modules are used when deploying complex resource stacks using Terraform. Attributes of resources created in modules are not exposed to the root module, so output files are used. Root module resources that are dependent of module-level resources must be configured to reference their attributes as exposed in the output files.

Before you begin

Before you begin, ensure you have completed lab5a.

Solution

The solution to this lab is in folder C:\azurelabs\solutions\lab5. Try to use this only as a last resort if you are struggling to complete the step-by-step processes.

Task 1. Review deployment of pre-defined infrastructure

In lab5a you deployed resources into the East US region. The code for this deployment consists of a root module and a network module. The network module deployed the prerequisites needed for the creation of virtual machines; the resource group, vnet, subnets etc. Examine main.tf at the root level to see how modules are referenced…

```
20  module "networking" {
21    source = "./network_module"
22  }
```

At the root module level, vm.tf contains code for the deployment of 2 virtual machines. These virtual machines have IIS installed which generates a default html page showing the virtual machine name. At present the VMs do not have public IP addresses and are therefore unavailable for public access.

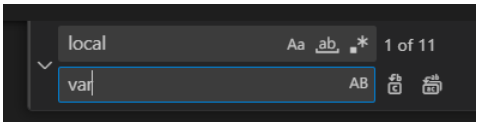
Task 2. Deploy an Application Gateway

1. Having deployed these resources, you are to provision an Application Gateway to distribute Http:80 requests across VM1 and VM2 using the provisioned Public IP address.

2. You are not to modify any existing code as this has been change-control approved. You are to update **agw.tf** to create 2 resources; an application gateway and an association between this and the network interfaces of the 2 VMs.
3. Parameters needed for the creation of these resources are within the network modules' **outputs.tf** file and the root modules' **variables.tf** file.
4. Successful completion of this task will be browsing to the public IP address and redirection to each of the 2 backend VMs in turn, the name of the VM being displayed in your web browser.
5. Attempt this lab yourself if you feel confident, otherwise use the step-by-step instructions below.
6. Browse to https://registry.terraform.io/providers/hashicorp/azurerm/4.16.0/docs/resources/application_gateway

7. Scroll down to find the sample code for an **azurerm_application_gateway** resource and copy this into agw.tf.

8. The sample code assumes the creation of a local list of variables. In our case though we are using a variables file and an outputs file from the networking module. Use the Find/Replace feature in VSC to replace all instances of 'local' with 'var' There should be 11 replacements.



9. The Resource group into which these resources will be deployed is created in the network_module and therefore attributes such as its name and location are not known at the root level. The outputs file in the network_module expose these attributes and must therefore be referenced.

10. For the azure_application_gateway.network resource block, Change..

```
resource "azurerm_application_gateway" "network" {
  name = "example-appgateway"
```

```
  resource_group_name = azurerm_resource_group.example.name
  location = azurerm_resource_group.example.location
```

to..

```
resource "azurerm_application_gateway" "network" {
  name = "myAppGateway"
  resource_group_name = module.networking.rg_name
  location = module.networking.rg_location
```

11. The same applies to subnet_ids and IP configuration so change..

```
gateway_ip_configuration {
  name = "my-gateway-ip-configuration"
  subnet_id = azurerm_subnet.frontend.id
}
```

to..

```
gateway_ip_configuration {
  name = "my-gateway-ip-configuration"
  subnet_id = module.networking.frontend_subnet_id
}
```

12. Also change the reference to the public IP address from ..

```
  frontend_ip_configuration {
    name = var.frontend_ip_configuration_name
    public_ip_address_id = azurerm_public_ip.example.id
  }
```

to…

```
frontend_ip_configuration {
    name = var.frontend_ip_configuration_name
    public_ip_address_id = module.networking.public_ip_address_id
  }
```

13. The backend_http_settings are currently…

```
backend_http_settings {
  name = var.http_setting_name
  cookie_based_affinity = "Disabled"
  path = "/path1/"
  port = 80
  protocol = "Http"
  request_timeout = 60
}
```

14. Scroll down the Terraform registry page to find the Argument Reference for this block and try to understand the values assigned. Cookie based affinity would cause all requests from a given endpoint to be directed to the same backend instance each time, necessary for stateful backends. The path parameter is used as the prefix for all HTTP requests and is optional. If there is no value then a path can be included in the url browsed to direct traffic to specific pages. No path will direct traffic to the default page for the site.

15. Remove the path argument as our VMs have a default page that will display the VM name. This will leave …

```
backend_http_settings {
  name = var.http_setting_name
  cookie_based_affinity = "Disabled"
  port = 80
  protocol = "Http"
  request_timeout = 60
}
```

16. Save agw.tf

Task 3. Configure nic associations

1. In order to associate the virtual machines as backends behind the application gateway, the azurerm_network_interface_application_gateway_backend_address_pool_association resource is used.

2. Open the link below, scroll down the last entry in Example Usage section to find an association example. Copy this into agw.tf….

```
resource "azurerm_network_interface_application_gateway_backend_address_pool_associ
  network_interface_id    = azurerm_network_interface.example.id
  ip_configuration_name   = "testconfiguration1"
  backend_address_pool_id = tolist(azurerm_application_gateway.network.backend_addr
}
```

https://registry.terraform.io/providers/hashicorp/azurerm/4.16.0/docs/resources/network_interface_application_gateway_backend_address_pool_association

3. Save agw.tf and run **terraform plan**. The following error message should be displayed…

   "Because azurerm_network_interface.example has "for_each" set, its attributes must be accessed on specific instances.
   For example, to correlate with indices of a referring resource, use: azurerm_network_interface.example[each.key]"

4. A map structure was used when creating the virtual machines (look at vm.tf for details). Therefore we must reference the same map structure when creating the associations between the vm nics and the gateway backend.

5. Replace the entire association code block with the one shown below (reduced font size used to prevent line-wrap issues)..

```
resource "azurerm_network_interface_application_gateway_backend_address_pool_association" "nic-assoc" {
 for_each = azurerm_network_interface.example
 network_interface_id    = each.value.id
 ip_configuration_name   = "${each.key}-ipconfig"
 backend_address_pool_id = one(azurerm_application_gateway.network.backend_address_pool).id
}
```

6. Save agw.tf
7. Use **terraform plan** to verify there are no issues with your code before using **terraform apply** followed by **yes** to deploy.
8. Once deployed (it may take 5-8 minutes so read the discussion about the association block below whilst waiting), open a web browser on your local machine and browse to the URL shown as the root modules' output.
9. Refresh your browser periodically until you see the name of the VM you are being directed to, VM1 or VM2. Refresh your browser several time and note that you are rotated across the 2 backends behind the application gateway.

Association code block discussion

Let's break down the code block:

for_each = azurerm_network_interface.example

for_each: This argument tells Terraform to create one instance of this resource for each instance in the provided map. In this case, the map is the one generated by the `azurerm_network_interface.nic` resource, which we previously defined with a `for_each` using a map of VM identifiers.

network_interface_id = each.value.id

network_interface_id: This attribute specifies the ID of the network interface to be associated. each.value.id: dynamically retrieves the ID of each network interface created in the azurerm_network_interface.example resource block in vms.tf

ip_configuration_name = "${each.key}-ipconfig"

ip_configuration_name: This specifies the name of the IP configuration within the network interface that you want to associate with the backend address pool. The name is dynamically generated using `**each.key**`, which corresponds to the keys from the original map used in `azurerm_network_interface.nic` (e.g., "VM1", "VM2"), appended with `-ipconfig`.

backend_address_pool_id = one(azurerm_application_gateway.network.backend_address_pool).id

backend_address_pool_id: This is the ID of the Application Gateway Backend Address Pool that the network interface will be associated with. The `one` function is used to assert that exactly one backend address pool is present within the `azurerm_application_gateway.network.backend_address_pool` resource. If there is exactly one pool, it retrieves its ID; if not, it will cause an error.

So, the `for_each` in this resource block is creating an association for each network interface that was created by the `azurerm_network_interface.nic` resource block, which was defined to create network interfaces for each entry in the provided map (e.g., "VM1" = "nic-vm1", "VM2" = "nic-vm2").

Task4. Lab Cleanup

1. Destroy all of your resources using **terraform destroy** and then entering **yes**.
2. Using the Azure portal, manually delete resource group **RG1**

****** Congratulations, you have completed this lab ******